ANGULAR FORMS: FORMULARIOS Y VALIDACIÓN

Formularios Template-Driven, directivas ngForm y ngModel, Reactive Forms, FormGroup, FormControl y FormArray, validadores de Angular y validadores personalizados

Apuntes revisados y ampliados

Documento elaborado a partir del fastbook original de Rodrigo Zavala

Contenido

1. Formularios Template-Driven 3

1.1 Qué son 3

1.2 ngForm 3

1.3 ngModel 3

1.4 ngModelGroup 3

1.5 ngModelChange 4

1.6 ngControl: referencias a un control 4

2. Reactive Forms 5

2.1 Qué son 5

2.2 FormGroup 5

2.3 FormControl 5

2.4 FormArray 5

2.5 Directivas de los formularios reactivos 6

3. Validators 7

3.1 Validadores incorporados 7

3.2 Ejemplos de uso 7

4. Custom Validators 8

5. Qué hemos aprendido 10

Introducción

Los formularios son la vía principal por la que el usuario introduce datos y los envía al servidor. Angular ofrece un sistema de formularios completo que permite crearlos, rastrear su estado y validarlos. En este tema se parte de lo más básico (formularios dirigidos por plantilla) para pasar después a los formularios reactivos, los validadores que trae Angular y, por último, cómo crear validadores propios.

Angular proporciona **dos enfoques** para construir formularios:

- **Template-Driven:** el formulario se define en la plantilla HTML mediante directivas.
- **Reactive Forms:** el formulario se define y controla desde el código TypeScript.

1. Formularios Template-Driven

1.1 Qué son

Son la forma más **simple y declarativa** de crear formularios. En lugar de definir el modelo de datos y la validación en el código, se apoyan en **directivas incorporadas en la plantilla HTML**. Son ideales para crear formularios rápidamente en aplicaciones sencillas.

Estas directivas son atributos que controlan y validan la entrada del usuario y permiten definir la estructura y las validaciones del formulario directamente en el HTML. Para usarlas hay que **importar FormsModule**, el módulo de formularios de Angular que las incluye.

1.2 ngForm

Crea el formulario en la plantilla y **rastrea el estado** del formulario y de sus controles (válido, modificado, tocado…). Con una referencia local se puede acceder a él desde la plantilla.

```
<form #myForm="ngForm">
  <!-- Contenido del formulario -->
</form>
```

1.3 ngModel

Enlaza un campo de entrada (*<input>*, *<textarea>* o *<select>*) con una propiedad del modelo de datos del componente. Con la sintaxis **[(ngModel)]** el enlace es **bidireccional** (two-way binding): cualquier cambio en el campo se refleja en el modelo y viceversa.

```
<input type="text" name="username" [(ngModel)]="user.username">
```

1.4 ngModelGroup

Agrupa campos relacionados bajo un nombre común dentro del modelo. Es útil para organizar y validar conjuntos de campos (p. ej. una dirección) dentro de un formulario grande.

```
<div ngModelGroup="address">
  <input type="text" name="street" [(ngModel)]="user.address.street">
  <input type="text" name="city"   [(ngModel)]="user.address.city">
</div>
```

1.5 ngModelChange

Evento que se emite cuando cambia el valor de un campo enlazado con ngModel. En el ejemplo, al cambiar *username* se ejecuta el método *onUsernameChange()* del componente.

```
<input type="text" name="username" [(ngModel)]="user.username"
       (ngModelChange)="onUsernameChange()">
```

1.6 ngControl: referencias a un control

Permite obtener una referencia a un campo enlazado con ngModel dentro de la plantilla, lo que resulta muy útil para mostrar mensajes de validación.

```
<input type="text" name="username" [(ngModel)]="user.username"
       #usernameField="ngModel">

<div *ngIf="usernameField.invalid && (usernameField.dirty || usernameField.touched)">
  <div *ngIf="usernameField.errors.required">El nombre de usuario es requerido.</div>
</div>
```

```
import { Component } from '@angular/core';

@Component({
  selector: 'app-form',
  templateUrl: './form.component.html',
})
export class FormComponent {
  usernameField: string = '';
}
```

Analizando el ejemplo:

- **#usernameField="ngModel":** crea una referencia local llamada *usernameField* vinculada al control del campo.
- **invalid, dirty, touched:** propiedades de estado del control; el error solo se muestra si el campo es inválido y el usuario ya lo ha modificado o tocado.
- **errors.required:** indica qué validación concreta ha fallado.

*Nota práctica (no aparece en el fastbook): la referencia #usernameField vive en la plantilla, así que no hace falta declararla en el TypeScript. Para que exista errors.required el input debe llevar el atributo required, y conviene usar usernameField.errors?.['required']. En Angular 17+ puede sustituirse *ngIf por el bloque @if.*

★ Lo más importante

► **Template-Driven** = formulario definido en el HTML con directivas · requiere **FormsModule**.

► **ngForm** crea el formulario y rastrea su estado · **ngModel** enlaza un campo con el modelo; **[(ngModel)]** = enlace bidireccional.

► **ngModelGroup** agrupa campos relacionados · **(ngModelChange)** reacciona a cambios de valor.

► **#ref="ngModel"** da acceso al control en la plantilla: *invalid*, *dirty*, *touched*, *errors*.

2. Reactive Forms

2.1 Qué son

Son una forma **más programática y potente** de manejar formularios: el formulario se define y controla desde **código TypeScript** en lugar de depender de la plantilla. Ofrecen más flexibilidad y control para formularios complejos y para gestionar validaciones.

*Nota práctica (no aparece en el fastbook): para usar sus directivas hay que importar **ReactiveFormsModule** (de @angular/forms) en el componente standalone o en el módulo.*

2.2 FormGroup

Clase que representa un **grupo de controles**; cada control se asocia a un campo del formulario. Se usa para crear la instancia del formulario y definir sus controles y validaciones.

```
import { FormGroup, FormControl, Validators } from '@angular/forms';

const myForm = new FormGroup({
  username: new FormControl('', Validators.required),
  email: new FormControl('', [Validators.required, Validators.email]),
});
```

2.3 FormControl

Representa un **campo de entrada individual** (texto, casilla de verificación, grupo de botones de opción...). Se usa para definir su **valor inicial** (primer parámetro) y sus **validaciones** (segundo parámetro).

```
import { FormControl, Validators } from '@angular/forms';

const usernameControl = new FormControl('', Validators.required);
```

2.4 FormArray

Representa un grupo de controles cuando se necesita una **colección dinámica** de elementos repetidos. Ejemplo: el usuario puede introducir uno o varios teléfonos y no se sabe cuántos; FormArray es una lista dinámica de controles que permite **añadir y quitar campos** según lo necesite.

```
import { FormArray, FormControl } from '@angular/forms';

const phoneNumbers = new FormArray([
  new FormControl('123-456-7890'),
  new FormControl('987-654-3210'),
]);
```

2.5 Directivas de los formularios reactivos

Directiva	Qué hace
[formGroup]	Enlaza un FormGroup definido en el componente con el <form> de la vista.
formControlName	Enlaza un FormControl concreto con un campo de la plantilla, para que Angular actualice su valor y lo valide.
formArrayName	Como formControlName, pero enlaza un FormArray con una lista de elementos de la plantilla.

```
<form [formGroup]="myForm" (ngSubmit)="onSubmit()">
  <input type="text"  formControlName="username">
  <input type="email" formControlName="email">
</form>
```

```
<form [formGroup]="myForm" (ngSubmit)="onSubmit()">
  <!-- Otros campos del formulario... -->

  <!-- FormArray para números de teléfono -->
  <div formArrayName="phoneNumbers">
    <!-- Bucle -->
    <div *ngFor="let phoneNumber of phoneNumbers.controls; let i = index">
      <input type="text" [formControlName]="i">
    </div>
  </div>

  <!-- Botón que lanza el evento submit del formulario (onSubmit) -->
  <button type="submit">Enviar</button>
</form>
```

Dentro de un FormArray cada control se identifica por su **índice**, por eso se usa *[formControlName]*="*i*". El evento **(ngSubmit)** ejecuta el método del componente al enviar el formulario.

Nota práctica (no aparece en el fastbook): phoneNumbers debe ser accesible desde la plantilla; lo habitual es un getter en el componente: get phoneNumbers() { return this.myForm.get('phoneNumbers') as FormArray; }.

Aspecto	Template-Driven vs Reactive Forms
Dónde se define	En la plantilla HTML · En el TypeScript
Módulo	FormsModule · ReactiveFormsModule
Enlace de campos	ngModel · formControlName / formArrayName
Ideal para	Formularios simples · Formularios complejos y dinámicos

★ Lo más importante

► **Reactive Forms** = formulario definido en TypeScript · más control y flexibilidad.

► **FormGroup** = grupo de controles · **FormControl** = un campo (valor inicial, validadores) · **FormArray** = lista dinámica de controles.

► En la vista: **[formGroup]** en el <form>, **formControlName** en cada campo, **formArrayName** en la lista.

► En un FormArray los controles se enlazan por índice: *[formControlName]*="*i*".

3. Validators

Angular trae una serie de **validadores ya hechos** para los casos más habituales, lo que acelera mucho el desarrollo. Lo más común es pasarlos como **segundo parámetro del FormControl**: uno solo o un array.

```
import { FormGroup, FormControl, Validators } from '@angular/forms';

const myForm = new FormGroup({
  username: new FormControl('', Validators.required),
  email: new FormControl('', [Validators.required, Validators.email]),
});
```

3.1 Validadores incorporados

Validador	Qué comprueba / para qué sirve
Validators.required	Que el campo no esté vacío ni sea nulo. Marca un campo como obligatorio.
Validators.minLength(n)	Que la longitud del valor sea ≥ n. Longitud mínima de un texto.
Validators.maxLength(n)	Que la longitud del valor sea ≤ n. Limita el número de caracteres.
Validators.pattern(p)	Que el valor coincida con una expresión regular (string o RegExp). Formatos concretos: emails, teléfonos...
Validators.email	Que el valor sea un email válido según una expresión regular predefinida.
Validators.min(n)	Que un valor numérico sea ≥ n. Valor mínimo.
Validators.max(n)	Que un valor numérico sea ≤ n. Valor máximo.
Validators.nullValidator	No valida nada: el campo siempre es válido. Útil para desactivar temporalmente la validación.
Validators.compose([...])	Combina varios validadores en una sola función de validación.
Validators.composeAsync([...])	Combina validadores asíncronos (devuelven observables), p. ej. comprobar en BD si un usuario ya existe.

3.2 Ejemplos de uso

```
import { Validators } from '@angular/forms';

const requiredValidator  = Validators.required;
const minLengthValidator = Validators.minLength(5);
const maxLengthValidator = Validators.maxLength(10);
const emailValidator     = Validators.email;
const minValidator       = Validators.min(18);
const maxValidator       = Validators.max(100);
const nullValidator      = Validators.nullValidator;

// Expresión regular que valida un email del tipo name@domain.xx
const emailPatternValidator =
  Validators.pattern(/^[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,4}$/);
```

```
import { Validators } from '@angular/forms';

const combinedValidator = Validators.compose([
  Validators.required,
  Validators.minLength(3)
]);

const combinedAsyncValidator = Validators.composeAsync([
  asyncValidator1, // Observable
  asyncValidator2  // Observable
]);
```

★ Lo más importante

► Los validadores van como **2.º parámetro del FormControl**: uno suelto o un **array**.

► **minLength/maxLength** → longitud de texto · **min/max** → valor numérico.

► **pattern** → expresión regular · **email** → formato de correo · **required** → obligatorio.

► **compose** combina validadores síncronos · **composeAsync** combina asíncronos (observables) · **nullValidator** no valida nada.

4. Custom Validators

Los validadores de Angular no cubren todas las necesidades, así que podemos crear los nuestros apoyándonos en las interfaces que proporciona Angular: **ValidatorFn**, **AbstractControl**, **ValidationErrors**...

Ejemplo: el campo *password* debe tener al menos **una letra mayúscula y un número**. Implementamos la función *customPasswordValidator*:

```
import { AbstractControl, ValidatorFn, ValidationErrors } from '@angular/forms';

// Definimos una función que actuará como nuestro validador personalizado
export function customPasswordValidator(): ValidatorFn {
  return (control: AbstractControl): ValidationErrors | null => {
    // Obtener el valor del campo de entrada
    const password = control.value;

    // Usar una expresión regular para verificar si contiene una letra mayúscula y un número
    const hasUpperCase = /[A-Z]/.test(password);
    const hasNumber = /\d/.test(password);

    // Comprobar si ambos criterios se cumplen
    if (!hasUpperCase || !hasNumber) {
      // Devolver un objeto con un error personalizado si no se cumple la validación
      return { customPassword: true };
    }

    // Si se cumplen los criterios, la validación es exitosa y devolvemos null
    return null;
  };
}
```

Analizando el ejemplo:

- **Función fábrica:** *customPasswordValidator()* devuelve una función de validación (*ValidatorFn*).
- **Valor del control:** la función interna recibe el *AbstractControl* y lee *control.value*.
- **Comprobación:** dos expresiones regulares verifican la mayúscula (*hasUpperCase*) y el número (*hasNumber*).
- **Error:** si falla algún criterio, devuelve un objeto de error personalizado *{ customPassword: true }*.
- **Éxito:** si se cumplen ambos, devuelve **null** (null = válido).

Solo queda usarlo dentro de un FormControl, igual que un validador de Angular:

```
import { Component } from '@angular/core';
import { FormGroup, FormControl, Validators } from '@angular/forms';
import { customPasswordValidator } from './custom-validators';
// Importamos nuestro validador personalizado

@Component({
  selector: 'app-my-form',
  templateUrl: './my-form.component.html',
})
export class MyFormComponent {
  myForm: FormGroup;

  constructor() {
    // Creamos un FormGroup utilizando new FormGroup y definimos los controles
    this.myForm = new FormGroup({
      password: new FormControl('', [Validators.required, customPasswordValidator()]),
      // Otros campos del formulario...
    });
  }
}
```

Los validadores personalizados son muy habituales: por ejemplo, para **comparar dos campos de contraseña**, comprobar con detalle **números de teléfono** o validar **campos que dependen de otro**.

Documentación oficial para ampliar: *angular.dev/guide/forms*.

★ Lo más importante

► Un validador personalizado es una función que devuelve un **ValidatorFn**: *(control: AbstractControl) => ValidationErrors | null*.

► Devuelve **null** si es válido · un **objeto de error** (p. ej. *{ customPassword: true }*) si no lo es.

► Se usa como cualquier otro: dentro del array del FormControl, **llamando** a la fábrica: *customPasswordValidator()*.

► Casos típicos: confirmar contraseña, teléfonos, campos dependientes de otro.

5. Qué hemos aprendido

Hemos visto que Angular ofrece dos maneras de construir formularios. Los **Template-Driven** se definen en el HTML con *FormsModule* y directivas como *ngForm*, *ngModel* (enlace bidireccional), *ngModelGroup* y *ngModelChange*, y permiten consultar el estado de cada control mediante referencias locales. Los **Reactive Forms** trasladan la definición al TypeScript con *FormGroup*, *FormControl* y *FormArray*, y se conectan a la vista con *[formGroup]*, *formControlName* y *formArrayName*. Después repasamos los **validadores incorporados** (required, min/maxLength, pattern, email, min/max, nullValidator, compose y composeAsync) y, por último, aprendimos a escribir **validadores personalizados** que devuelven *null* o un objeto de error. Con esto ya se pueden crear formularios completos y validados; el siguiente paso es practicar combinándolos y consultar la documentación oficial para casos avanzados como validadores asíncronos o validaciones entre campos.

¡Enhorabuena! Documento completado